

Assignment No -2

Name : Atharv Gade

Roll No: COB413

Title: Write a program to implement Parallel Bubble Sort and Merge sort using OpenMP. Use existing algorithms and measure the performance of sequential and parallel algorithms.

Source Code with Output :

```
import time
import random

import concurrent.futures

def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr

def parallel_bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        first = i % 2
        with concurrent.futures.ThreadPoolExecutor() as executor:
            futures = []
            for j in range(first, n - 1, 2):
                futures.append(executor.submit(swap_if_needed, arr, j))
            for future in concurrent.futures.as_completed(futures):
                future.result()
    return arr

def swap_if_needed(arr, j):
    if arr[j] > arr[j+1]:
        arr[j], arr[j+1] = arr[j+1], arr[j]

def merge(arr, left, right):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left_half = merge(arr[:mid], left, mid)
    right_half = merge(arr[mid:], mid, right)
    return
```

```
merge_sorted(left_half, right_half) def
```

```
merge_sorted(left, right):
```

```
    result = []    i = j = 0    while i <
len(left) and j < len(right):    if left[i]
< right[j]:
result.append(left[i])    i += 1
else:
result.append(right[j])    j +=
1    result.extend(left[i:])
result.extend(right[j:])    return
result
```

```
def parallel_merge_sort(arr):
```

```
    if len(arr) <= 1:    return arr    mid = len(arr) // 2    with
concurrent.futures.ThreadPoolExecutor() as executor:
left_future = executor.submit(parallel_merge_sort, arr[:mid])
right_future = executor.submit(parallel_merge_sort, arr[mid:])
left = left_future.result()    right = right_future.result()    return
merge_sorted(left, right)
```

```
def measure_time(sort_function, arr):
```

```
start_time = time.time()    sorted_arr =
sort_function(arr)    end_time = time.time()
return sorted_arr, end_time
- start_time
```

```
def main():    n = 100    print(f"Total
number of elements: {n}")
```

```
    arr = list(range(n, 0, -1))    print("Storing
elements in descending order...\n")    print("Actual
list:")
```

```
print(arr)
```

```
print("\nRunning Sequential Bubble Sort...")
arr_copy = arr.copy()    sorted_arr, time_taken =
measure_time(bubble_sort, arr_copy)
print("Sorted list (Bubble Sort):")    print(sorted_arr)
print(f'Execution Time: {time_taken:.6f} seconds')
```

```
print("\nRunning Parallel Bubble Sort...")
arr_copy = arr.copy()    sorted_arr, time_taken =
measure_time(parallel_bubble_sort, arr_copy)
print("Sorted list (Parallel Bubble Sort):")    print(sorted_arr)
print(f'Execution Time: {time_taken:.6f} seconds')
```

```
print("\nRunning Sequential Merge Sort...")
arr_copy = arr.copy()    sorted_arr, time_taken = measure_time(lambda arr: merge(arr,
0, len(arr)-1), arr_copy)
print("Sorted list (Merge Sort):")    print(sorted_arr)
print(f'Execution Time: {time_taken:.6f} seconds')
```

```
print("\nRunning Parallel Merge Sort...")
arr_copy = arr.copy()    sorted_arr, time_taken =
measure_time(parallel_merge_sort, arr_copy)
print("Sorted list (Parallel Merge Sort):")    print(sorted_arr)
print(f'Execution Time: {time_taken:.6f} seconds')
```

```
if __name__ == "__main__":    main()
```

Output :

```
Total number of elements: 100
Storing elements in descending order...
```

Actual list:

```
[100, 99, 98, 97, 96, 95, 94, 93, 92, 91, 90, 89, 88, 87, 86, 85, 84, 83, 82, 81, 80, 79, 78, 77, 76, 75, 74, 73, 72,
```

71, 70, 69, 68, 67, 66, 65, 64, 63, 62, 61, 60, 59, 58, 57, 56, 55, 54, 53, 52, 51, 50, 49, 48, 47, 46, 45, 44, 43, 42, 41, 40, 39, 38, 37, 36, 35, 34, 33, 32, 31, 30, 29, 28, 27, 26, 25, 24, 23, 22, 21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]

Running Sequential Bubble Sort...

Sorted list (Bubble Sort):

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100]

Execution Time: 0.000758 seconds

Running Parallel Bubble Sort...

Sorted list (Parallel Bubble Sort):

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100]

Execution Time: 0.266036 seconds

Running Sequential Merge Sort...

Sorted list (Merge Sort):

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100]

Execution Time: 0.000160 seconds

Running Parallel Merge Sort...

Sorted list (Parallel Merge Sort):

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100]

Execution Time: 0.053550 seconds